

Process Creation

2026 Semester 1 SOFTENG 370: Operating Systems

Talia Xu

Lecture 5

2.0.0

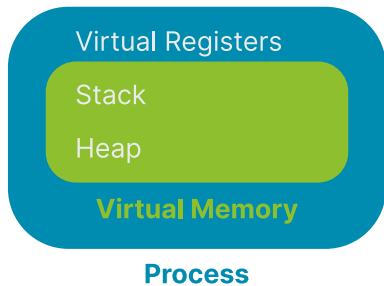
Based on original slides by Professor Jon Eyolfson.

This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/)

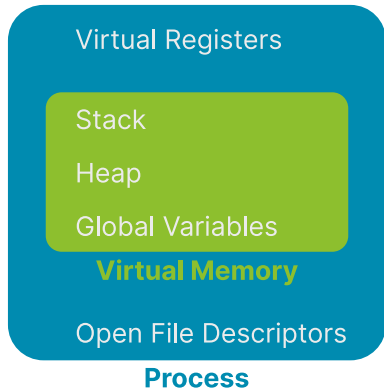
Program and Process

A **program** consists of static code and data, e.g., on the disk.

A **process** is an instance of a program (at any time there may be 0 or more instances of a program running, e.g., a user may run multiple concurrent shells).



We Can Add More to a Process



A Process Control Block (PCB) Contains All Information

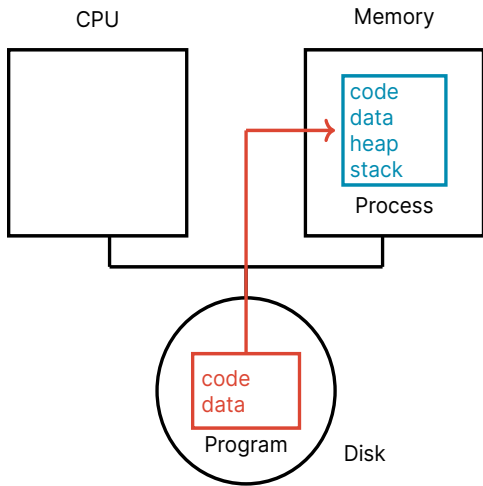
Specifically, in Linux, this is the [task_struct](#) you can browse on [GitHub](#)

It contains:

- Process ID (PID)
- Process state
- CPU registers
- Scheduling information
- Memory management information
- I/O status information
- Any other type of accounting information

Each process gets a unique process ID (pid) to keep track of it

Process Creation



Process Creation

OS allocates internal data structures (PCB)

OS allocates an address space

OS opens basic files (STDIN, STDOUT, STDERR)

OS initializes CPU registers

Program, Process, Thread

A **program** is on-disk application, consisting of code and data; programs become a process when they are executed

A **process** is a running instance of a program. A process starts with a single thread of execution and an address space.

A process can launch multiple **threads** of execution in the same address space. Each thread receives its own stack but they share global data, code, and heap.

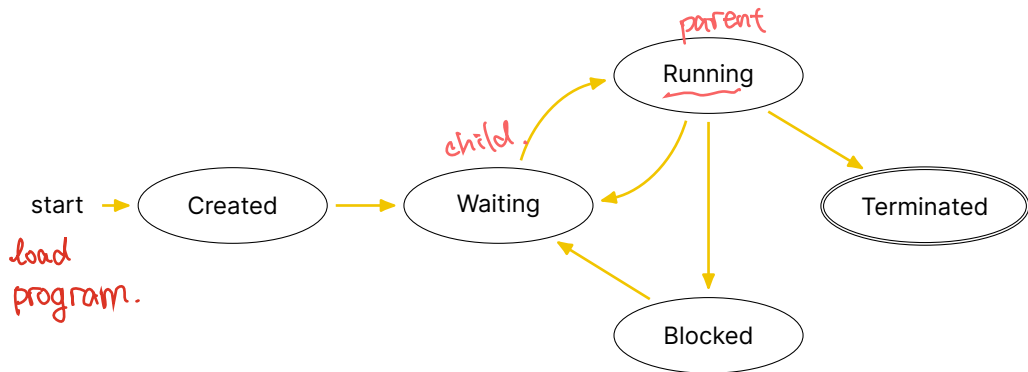
OS provides process abstraction

When the user executes a program, the OS creates a process

OS time-shares CPU across multiple processes

OS scheduler picks one of the executable processes to run

Process State Diagram (You Could Rename Waiting to Ready)



We Could Create Processes from Scratch

We create the process control block and load the program into memory
(this is what Windows does) *PID. info. memory state. / state.
register. scheduling.*

Unix decomposes process creation into more flexible abstractions

Instead of Creating a New Process, We Could Clone It

Pause the currently running process, and copy its PCB into a new one

This will reuse all of the information from the process, including variables!

Distinguish between the two processes with a parent and child relationship
old new.

fork Creates a New Process, A Copy of the Current One

(system call)

int fork(void) as the following API:

- Returns the process ID of the newly created child process
 - 1: on failure
 - 0: in the child process
 - >0: in the parent process

PID of child.

sep. memory.

There are now 2 processes running

Note: they can access the same variables, but they're separate
Operating system does "copy on write" to maximize sharing

fork Examples

```
#include <stdio.h>
#include <unistd.h>
```

```
int main() { ← .
```

```
    int value = 2;
```

```
    if (fork() == 0) ←
```

```
        printf("Child\n");
```

```
    else
```

```
        printf("Parent\n");
```

```
    //printf("Hello world, %d\n", value);
```

```
    return 0;
```

```
}
```

1 process (parent).

2 processes w/ evaluate if condition.

parent: PID of child (not 0).

child = 0.

child
start executing
here.

child →

parent →

fork Examples

```
int main() {  
    int value = 2;  
    if (fork() == 0)  
        printf("Hello world\n");  
    else  
        printf("Hello world again\n");  
    return 0;  
}
```

order ?

↳ arbitrary.

1000 times.

either parent or
child could print
first.

↳ parent : running state
child = waiting state.
(after
creation)

fork Examples

```
#include <stdio.h>
#include <unistd.h>

int main() {
    int value = 2;
    int pid = getpid();
    fork();
    value = 3;
    printf("Hello, world, %d, %lx!\n", value, (long int)&value);
    return 0;
}
```

2 times .

syscall

execve Replaces the Process with Another Program, and Resets

execve has the following API:

- pathname: Full path of the program to load
- argv: Array of strings (array of characters), terminated by a null pointer
Represents arguments to the process
- envp: Same as argv
Represents the environment of the process
- Returns an error on failure, does not return if successful

execve-example.c Turns the Process into ls

```
int main(int argc, char *argv[]) {  
    → printf("I'm going to become another process\n"); ←  
    char *exec_argv[] = {"ls", NULL};  
    char *exec_envp[] = {NULL};  
    int exec_return = execve("/usr/bin/ls", exec_argv, exec_envp); ← i execute "ls".  
    if (exec_return == -1) {  
        exec_return = errno;  
        perror("execve failed");  
        return exec_return;  
    }  
    → printf("Hello world!\n"); ←  
    return 0;  
}
```

new program

never executes except failure.